Understanding

AVR Interrupts

Amer Iqbal Qureshi | Mikrotronics Pakistan

Understanding AVR Interrupts

Before learning how to use interrupt-driven serial communication, it is important to understand what an interrupt actually is.

An **interrupt** is a hardware mechanism that temporarily pauses the normal execution of a program so that the processor can immediately respond to an important event. Once the event has been handled, the processor automatically returns to the exact point where it left off and continues executing the original program. This entire process happens in only a few microseconds.

A Real-World Analogy

Imagine you are reading a book. Everything is peaceful until the telephone rings. Instead of continuing to read, you

1. Place a bookmark in the book.
2. Answer the phone.
3. Finish the conversation.
4. Return to the exact page where you stopped reading.

You do not start reading from the beginning again. You continue exactly where you left off. The AVR processor behaves in exactly the same way. The currently executing instruction is temporarily interrupted, the processor handles the event, and then execution resumes as though nothing happened.

Normal Program Execution

Without interrupts, the processor simply executes one instruction after another.

```
Instruction 1
↓
Instruction 2
↓
Instruction 3
↓
Instruction 4
↓
Instruction 5
```

Nothing can interrupt this sequence.

Program Execution With Interrupts

Now suppose a serial character arrives while the CPU is executing Instruction 3.

The execution sequence becomes

```
Instruction 1
↓
Instruction 2
```

```
↓
Instruction 3
↓
USART Interrupt
↓
Interrupt Service Routine (ISR)
↓
Return
↓
Instruction 4
↓
Instruction 5
```

Notice that the interrupted instruction is completed before the processor jumps to the interrupt service routine. After the ISR finishes, the CPU automatically resumes execution at the next instruction. To the application, the interruption appears almost instantaneous.

What Happens Inside the AVR?

When an interrupt occurs, the AVR hardware performs several operations automatically.

1. The current instruction finishes.
2. The address of the next instruction (the Program Counter) is pushed onto the stack.
3. Global interrupts are temporarily disabled.
4. The processor jumps to the appropriate Interrupt Service Routine (ISR).
5. The ISR executes.
6. The RETI (Return from Interrupt) instruction restores the saved Program Counter.
7. The main program continues exactly where it stopped.

The programmer does not need to write any code to save or restore the Program Counter. The AVR hardware performs these operations automatically.

Interrupt Vectors

Every interrupt source has a fixed location in program memory called an **interrupt vector**. When an interrupt occurs, the processor jumps to the corresponding vector.

For example,

Interrupt Source	Interrupt Vector
Reset	Reset Vector
External Interrupt 0	INT0 Vector
Timer0 Overflow	TIMER0_OVF Vector
USART RX Complete	USART_RX Vector
USART Data Register Empty	USART_UDRE Vector
USART TX Complete	USART_TX Vector

Each vector points to a specific Interrupt Service Routine.

Think of the vector table as a directory that tells the processor where to go when a particular event occurs.

Interrupt Service Routine (ISR)

The code that executes when an interrupt occurs is called an **Interrupt Service Routine**, or **ISR**. In traditional AVR programming, an ISR is declared using the `ISR()` macro.

For example, the receive interrupt is written as

```
ISR(USART_RX_vect)
{
    uint8_t data = UDR0;

    // Process received character
}
```

Whenever a character arrives, this function is called automatically by the processor. The main program does not need to call it explicitly, provided interrupts are enabled on the USART.

Why the SDK Hides This Complexity

Although writing ISRs directly provides maximum flexibility, it also requires a thorough understanding of AVR registers, interrupt vectors, and interrupt configuration.

A beginner must remember to:

1. Enable the correct interrupt.
2. Write the correct ISR.
3. Read the appropriate hardware register.
4. Clear interrupt flags when necessary.
5. Avoid lengthy operations inside the ISR.

The MikroDuino SDK hides most of this complexity. Instead of writing low-level interrupt code, the programmer works with simple library functions while the SDK manages the hardware details behind the scenes. This allows beginners to benefit from interrupt-driven communication without becoming overwhelmed by the underlying implementation.

Later in this chapter, we will see how the SDK enables interrupt-driven USART communication with only a few lines of code.

Enabling Global Interrupts

Even if an interrupt source is enabled, the AVR processor will ignore all interrupts unless **Global Interrupts** are enabled.

This is controlled by the **I (Interrupt Enable)** bit in the AVR Status Register (`SREG`).

The simplest way to enable global interrupts is

```
sei();
```

To temporarily disable all interrupts

```
cli();
```

These two functions are provided by the AVR standard library and are used in almost every interrupt-driven application.

Fortunately, many MikroDuino SDK modules enable the required interrupts automatically during initialization, so in many cases the application programmer does not need to manipulate these settings directly.

Good Practices for Writing ISRs

Interrupt Service Routines should always be short and efficient.

A good ISR should:

- Read or write the required hardware registers.
- Save the received data to a buffer.
- Set a flag if necessary.
- Return immediately.

Avoid performing lengthy operations such as:

- Printing messages to the serial port.
- Calling delay functions.
- Executing complex calculations.
- Accessing slow peripherals.
- Waiting inside loops.

A common rule among embedded engineers is:

Do the minimum amount of work inside the ISR, and let the main program perform the rest.

Following this principle keeps the system responsive and reduces the risk of missed interrupts.

The Role of Interrupts in the MikroDuino USART Library

The USART library uses interrupts to make serial communication both efficient and reliable.

Instead of forcing the application to repeatedly poll the hardware, the SDK can:

- Capture incoming characters immediately using the RX Complete interrupt.
- Store received data in a circular receive buffer.
- Transmit outgoing data using the Data Register Empty (UDRE) interrupt.
- Notify the application when transmission is complete using the **TX Complete** interrupt.

From the programmer's perspective, serial communication appears almost effortless. While your application continues to read sensors, update displays, or control motors, the USART driver quietly handles the movement of data in the background.

Now we move from interrupt theory into the **first practical USART interrupt**. This is the most important interrupt in serial communication because nearly every embedded application needs to receive data reliably.

RX Complete Interrupt

The **RX Complete Interrupt** is generated whenever the USART successfully receives an entire character. As soon as the Stop Bit has been verified, the USART hardware performs two operations automatically:

- Copies the received byte into the **UDR0** register.
- Sets the **RXC0 (Receive Complete)** flag.

If the **RX Complete Interrupt** is enabled, the processor immediately executes the receive interrupt service routine. Unlike polling, the application does not need to continuously ask whether a character has arrived. The hardware notifies the CPU automatically.

Receiving a Character

Suppose a computer sends the letter

```
A
```

The sequence of events inside the ATmega328P is

```
PC
↓
TX Pin
↓
USART Receiver
```

```
↓
Shift Register
↓
UDR0
↓
RXC0 Flag Set
↓
RX Complete Interrupt
↓
Interrupt Service Routine
↓
Character Stored in Receive Buffer
↓
Main Program Continues
```

Notice that the application never misses the incoming character because the interrupt responds almost immediately after reception.

What Happens Inside the ISR?

The interrupt service routine should perform only a few simple operations.

Typically, it:

1. Reads the received byte from `UDR0`.
2. Stores the byte in a software receive buffer.
3. Updates the buffer index.
4. Returns immediately.

Conceptually, the ISR looks like this.

```
ISR(USART_RX_vect)
{
    uint8_t data = UDR0;

    rxBuffer[head] = data;

    head++;

    if(head >= BUFFER_SIZE)
        head = 0;
}
```

This example is simplified for explanation purposes.

The actual MikroDuino SDK implementation performs additional checks to prevent buffer overflow and maintain data integrity.

Why Read UDR0 Immediately?

One of the first tasks performed by the ISR is reading the `UDR0` register.

This is extremely important. The ATmega328P contains only a limited hardware receive buffer. If another character arrives before the previous one has been removed from `UDR0`, the hardware can no longer store the new character.

This results in a **Data OverRun (DOR0)** error.

Reading `UDR0` immediately allows the USART hardware to receive the next character without delay.

Software Receive Buffer

The hardware USART can hold only a very small amount of received data. To overcome this limitation, the MikroDuino SDK maintains a **software receive buffer** in RAM.

Instead of forcing the application to process each character immediately, the interrupt stores incoming data inside this buffer.

```
Incoming Characters
↓
USART Hardware
↓
RX Interrupt
↓
Software Receive Buffer
↓
Application Reads Later
```

The application and the interrupt now work independently.

- The interrupt receives characters.
- The application processes characters whenever convenient.

This greatly improves reliability.

Circular Buffer Operation

The receive buffer behaves like a circular queue. Suppose the buffer contains eight bytes.

```
+-----------------------------------+
| A | B | C | D | E |   |   |   |
+-----------------------------------+
          ↑               ↑
        Tail            Head
```

The **Head** points to the next free location where the interrupt stores incoming data. The **Tail** points to the next character waiting to be read by the application.

When the application calls

```
USART0.read();
```

the character at the Tail position is returned. The Tail pointer then advances to the next character. Likewise, every new interrupt advances the Head pointer. When either pointer reaches the end of the array, it wraps around to the beginning.

This is why it is called a **circular buffer**.

Why Use a Circular Buffer?

Without a software buffer, the application would need to process every received character immediately.

Consider this loop.

```
while(true)
{
    readSensors();

    updateLCD();

    calculatePID();

    saveEEPROM();
}
```

Suppose the user sends

```
HELLO
```

while the processor is busy updating the LCD. Several characters could arrive before the application checks the USART. With polling, some of these characters may be lost.

With interrupt-driven reception, every character is captured immediately and stored safely inside the circular buffer. Later, the application simply reads the buffered characters.

The SDK Advantage

Without the MikroDuino SDK, the programmer must write all of the following:

- The interrupt service routine.
- The receive buffer.
- Head and tail pointers.
- Overflow detection.
- Buffer wrap-around logic.
- Buffer synchronization.

The application code quickly becomes longer than the actual program being developed.

The MikroDuino SDK performs all of these tasks internally.

From the application's perspective, receiving data remains remarkably simple.

```
if (USART0.available())
{
    char ch = USART0.read();
}
```

Even though interrupts and circular buffers are working behind the scenes, the programmer interacts with a clean and intuitive interface.

This is one of the major advantages of using the SDK instead of programming the USART registers directly.

Polling vs. Interrupt Reception

Consider two applications receiving the same stream of characters.

Polling

```
Main Program
↓
Check USART
↓
No Character
↓
Continue
↓
Check Again
↓
Character Arrives
↓
Read Character
```

The processor repeatedly checks the USART, even when no data is present.

Interrupt-Driven Reception

```
Main Program Running
↓
Character Arrives
↓
RX Complete Interrupt
↓
Character Stored in Buffer
↓
Main Program Continues
↓
Application Reads Character Later
```

The processor spends almost no time monitoring the USART.

Instead, it concentrates on running the application while the interrupt system handles reception automatically.

Buffer Overflow

Although circular buffers are extremely efficient, they are not unlimited.

Suppose the receive buffer can store 64 characters. If the application never reads the received data, eventually the buffer becomes full. When additional characters arrive, one of two things must happen.

- The new characters are discarded.
- The oldest characters are overwritten.

The MikroDuino SDK protects against this situation by monitoring the buffer status and providing functions to determine whether data is available. As the application developer, you should regularly process incoming data to prevent the receive buffer from filling completely during periods of heavy communication.

Best Practice

Think of the interrupt as a collector, not a processor.

Its job is simply to collect incoming bytes as quickly as possible and place them into the receive buffer. The main application is responsible for interpreting those bytes, assembling complete messages, and performing any required processing. Keeping these responsibilities separate results in faster, more reliable, and easier-to-maintain software.

The Data Register Empty (UDRE) Interrupt

Receiving data is only one half of serial communication. Sooner or later, every application needs to transmit information.

In the previous chapter, we learned that functions such as `USART0.write("Hello World");`

appear to send data immediately. However, this is not exactly what happens inside the microcontroller. The USART hardware can transmit **only one byte at a time**. Each byte must first be copied into the **USART Data Register (UDR0)** before it can be shifted onto the TX pin.

While one byte is being transmitted, the processor must wait before loading the next byte. This waiting is the purpose of the **UDRE (USART Data Register Empty)** flag.

What is UDRE?

The **UDRE** flag indicates whether the transmit data register is ready to accept another byte.

When

```
UDRE = 1
```

the register is empty. A new byte may be written to `UDR0`.

When

```
UDRE = 0
```

the register is still busy. The processor must wait.

Polling Transmission

Consider sending the word

```
HELLO
```

using polling.

The processor performs the following sequence repeatedly.

```
Wait for UDRE
↓
Load UDR0
↓
Hardware Transmits Byte
↓
Wait Again
↓
Load Next Byte
↓
Repeat
```

This process works perfectly. However, notice that the CPU spends much of its time waiting. If the baud rate is low, the waiting becomes even longer.

For example,

- At **9600 baud**, transmitting one character takes approximately **1 millisecond**.
- During this time, a 16 MHz AVR executes **thousands of instructions**.

Those processor cycles are effectively wasted.

Interrupt-Driven Transmission

Instead of repeatedly checking the UDRE flag, the USART hardware can notify the processor automatically. Whenever the transmit register becomes empty, the USART generates a **Data Register Empty Interrupt**.

The sequence becomes

```
Main Program Running
↓
UDR0 Becomes Empty
↓
UDRE Interrupt
↓
Load Next Byte
↓
Return
↓
Main Program Continues
```

The processor never waits. It only spends a few microseconds loading the next character before returning to the application.

Why Is This Better?

Imagine transmitting a long message.

```
MikroDuino SDK USART Library
```

Using polling, the processor repeatedly waits for every character. Using interrupts, the processor simply places the message into a transmit buffer and immediately continues executing the rest of the application.

The USART hardware and interrupt system handle the transmission automatically. To the application programmer, it appears as though transmission occurs "in the background."

The Software Transmit Buffer

Just as received characters are stored in a receive buffer, characters waiting to be transmitted are stored inside a **transmit buffer**. Conceptually,

```
Application
↓
write()
↓
Transmit Buffer
↓
UDRE Interrupt
↓
UDR0
↓
```

```
TX Pin
```

Whenever the USART finishes accepting one byte, the interrupt automatically loads the next byte from the transmit buffer. The application does not need to monitor the hardware.

How the SDK Uses the UDRE Interrupt

One of the strengths of the MikroDuino SDK is that it completely hides this process. When the application executes

```
USART0.write("Temperature = ");
```

the library

1. Copies the characters into the transmit buffer.
2. Enables the UDRE interrupt.
3. Returns immediately.

From this point onward, every time the USART becomes ready, the interrupt automatically transfers another character from the software buffer into `UDR0`. When the last character has been loaded, the library simply disables the UDRE interrupt until more data needs to be transmitted.

The application never needs to manage this process.

Conceptual ISR

A simplified transmit interrupt might look like this.

```
ISR(USART_UDRE_vect)
{
    if(buffer is not empty)
    {
        UDR0 = next character;
    }
    else
    {
        disable UDRE interrupt;
    }
}
```

The actual SDK implementation is considerably more sophisticated, handling circular buffers, synchronization, and buffer management internally.

Continuous Background Transmission

Suppose the application is controlling a robot.

```
Read Sensors
↓
```

```
Calculate Motor Speed
↓
Update OLED Display
↓
Transmit Debug Information
↓
Read Sensors
```

Without interrupts, serial transmission forces the CPU to stop repeatedly while waiting for the USART. With the UDRE interrupt, debug messages are transmitted automatically while the application continues controlling the robot.

This is known as **background transmission** or **asynchronous transmission**.

Polling vs. UDRE Interrupt

Polling

```
CPU
↓
Wait
↓
Send Byte
↓
Wait
↓
Send Byte
↓
Wait...
```

Large portions of processor time are spent waiting.

Interrupt-Driven Transmission

```
Application
↓
Fill Transmit Buffer
↓
Continue Working
↓
USART Requests Next Byte
↓
Interrupt Sends Byte
↓
Application Never Stops
```

Notice that the application no longer waits for the serial hardware.

Buffer Management

Like the receive buffer, the transmit buffer has a fixed capacity. If the application attempts to transmit data faster than the USART can send it, the transmit buffer eventually becomes full.

At this point, the library may

- wait until space becomes available,
- reject additional data,
- or use another policy depending on the library configuration.

Fortunately, for most applications, the transmit buffer is emptied continuously by the UDRE interrupt, making buffer overflow uncommon.

Why Not Use the TX Complete Interrupt?

Many beginners assume that transmission should be handled using the **TX Complete** interrupt. In reality, professional serial drivers almost always use the **UDRE interrupt**.

The reason is simple. The UDRE interrupt occurs **as soon as the transmit register becomes empty**, allowing the next byte to be loaded immediately. The USART hardware continues shifting the previous byte while the next byte is already waiting.

This keeps the transmitter operating continuously with no unnecessary gaps between characters. The TX Complete interrupt occurs **too late**. By the time it is generated, the previous frame has already finished transmitting, introducing a small delay before the next byte can be loaded.

For maximum throughput, the UDRE interrupt is therefore the preferred choice for continuous serial transmission.

The SDK Advantage

Without the MikroDuino SDK, implementing interrupt-driven transmission requires writing

- a transmit interrupt service routine,
- a circular transmit buffer,
- head and tail pointer management,
- interrupt enable and disable logic,
- synchronization between the main program and the interrupt,
- and careful protection against race conditions.

This is a considerable amount of code before a single application-specific feature has been implemented. The MikroDuino SDK performs all of these tasks internally. From the programmer's perspective, transmitting data remains as simple as

```
USART0.writeLine("System Ready");
```

while the SDK efficiently manages buffering, interrupt handling, and background transmission behind the scenes.

The TX Complete Interrupt

The TX Complete Interrupt occurs when the USART has completely finished transmitting a frame.

This means:

- All data bits have been shifted out.
- The parity bit (if enabled) has been transmitted.
- The final Stop Bit has completely left the TX pin.

Only after this point does the USART set the TXC0 (Transmit Complete) flag and generate the TX Complete interrupt.

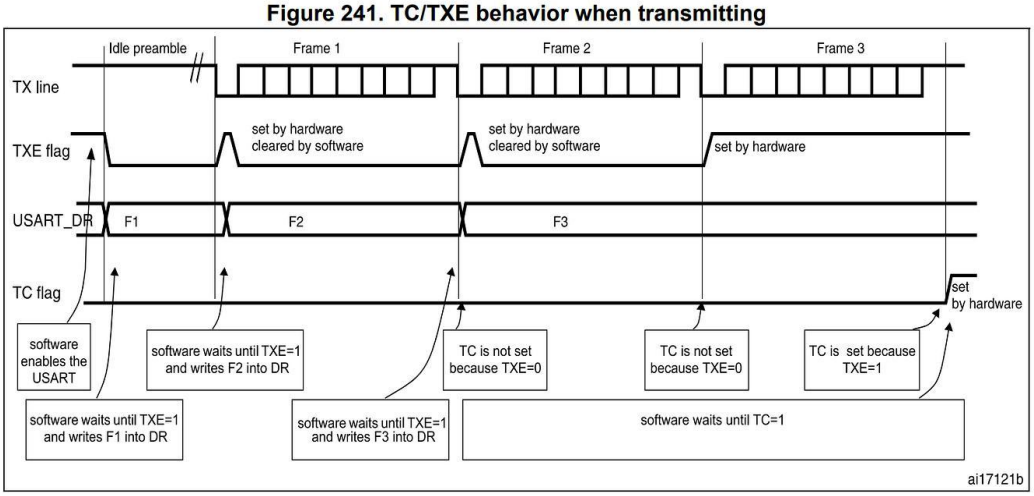
UDRE vs. TX Complete

This distinction is extremely important.

Interrupt	Meaning
UDRE	"You may load the next byte now."
TX Complete	"The last bit has physically left the TX pin."

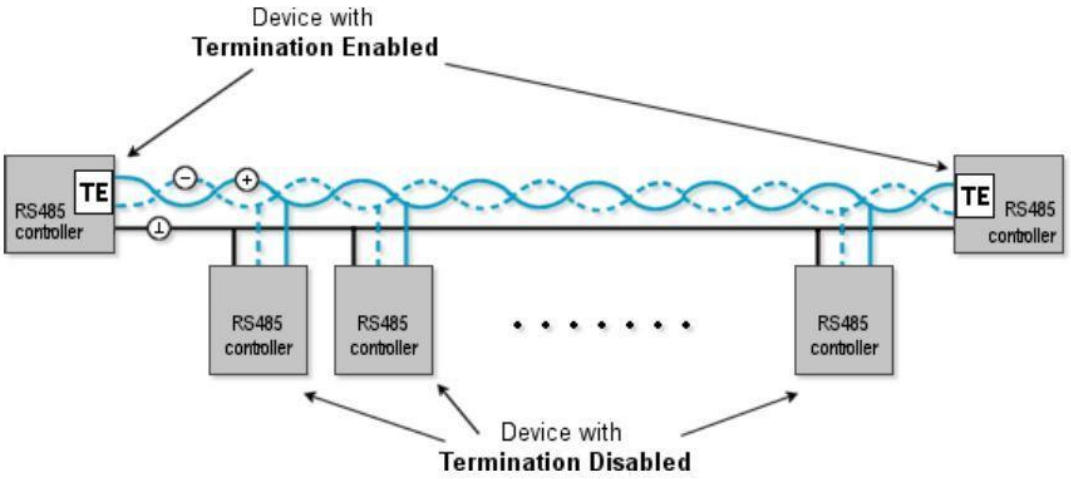
Think of it this way:

Figure 241. TC/TXE behavior when transmitting



Why Does TX Complete Exist?

For ordinary PC communication, you rarely need this interrupt. However, many industrial systems use RS-485 instead of simple UART wiring.



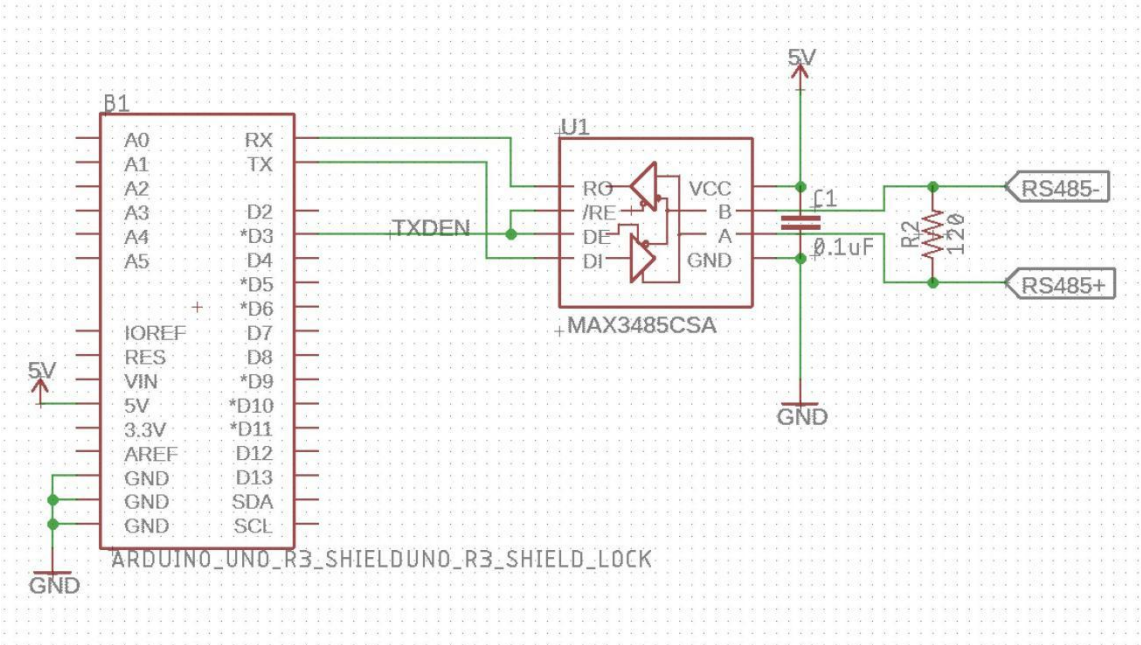
RS-485 is usually half-duplex, meaning:

- Only one device may transmit at a time.
- The transmitter must be enabled before sending data.
- The transmitter must be disabled immediately after transmission finishes.

If you disable the transmitter too early, the final character becomes corrupted. The TX Complete interrupt solves this problem perfectly.

A Practical Example

Suppose an RS-485 transceiver has a direction-control pin called DE (Driver Enable).



Before transmitting:

- DE = HIGH → transmitter enabled.

- Send data through USART.
- Wait until transmission is truly finished.
- DE = LOW → return to receive mode.

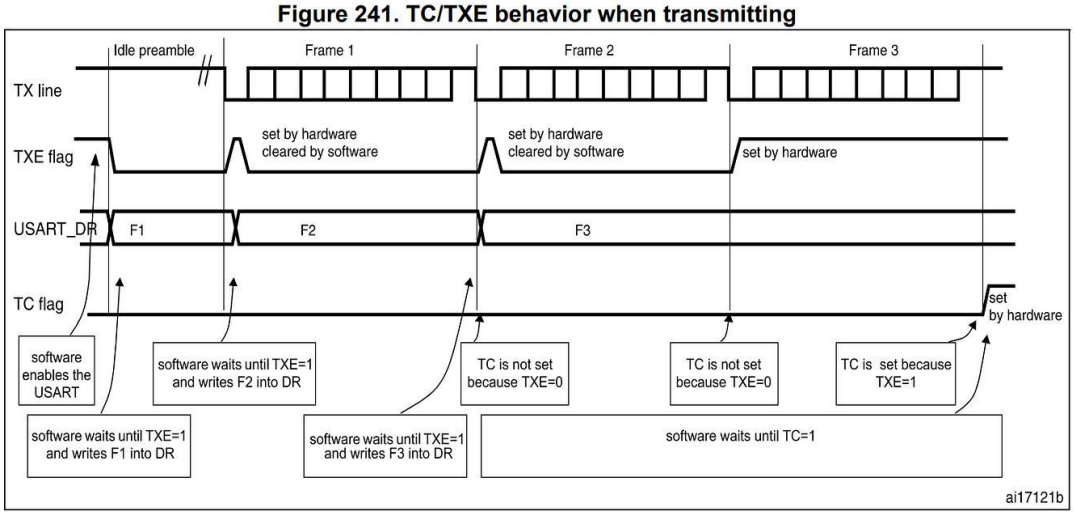
The critical question is:

"How do we know when transmission is truly finished?"

The answer is the TX Complete Interrupt.

Transmission Timeline

Figure 241. TC/TXE behavior when transmitting



Notice that the transmitter remains enabled until the last stop bit has completely exited the TX pin.

Conceptual ISR

A simplified TX Complete interrupt routine looks like this:

The ISR performs only one very small operation and returns immediately.

When Should You Use TX Complete?

Use TX Complete when:

- Controlling RS-485 direction pins.
- Switching between transmit and receive modes.
- Generating a signal after the final byte has left the UART.
- Measuring exact transmission completion time.

Do NOT use TX Complete for:

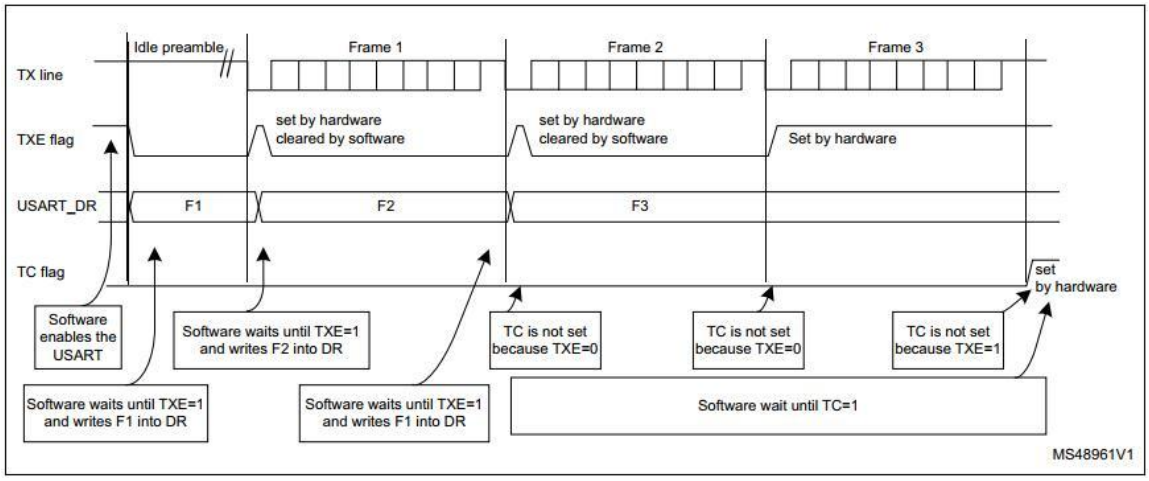
- Continuous character transmission.
- Buffered serial output.
- Normal terminal communication.

- High-throughput data streams.

Why UDRE Is Still the Main Transmission Interrupt

Professional serial drivers almost always transmit data using UDRE interrupts, not TX Complete interrupts.

Here's why:



Using TX Complete for every byte would create tiny gaps between characters, reducing throughput. Using UDRE keeps the transmitter continuously busy. Therefore, the typical design is:

Interrupt	Purpose
RX Complete	Receive characters.
UDRE	Transmit buffered data.
TX Complete	Detect final transmission completion.

How the MikroDuino SDK Uses TX Complete

Your SDK can use the TX Complete interrupt to:

- Notify the application that all queued data has been sent.
- Support RS-485 communication.
- Trigger user callbacks after transmission finishes.
- Implement advanced asynchronous communication patterns.

This is a feature that many beginner-oriented serial libraries simply do not expose.

The Complete USART Interrupt Picture

```
/*****************************************************************
**********
* USART Interrupt Example
*
* Demonstrates:
```

```
* - USART initialization
* - RX Complete Interrupt
* - Circular receive buffer
* - Echo received characters
* - Processing complete commands
*
* Hardware:
* ATmega328P
* Baud: 9600
*****************************************************************
*********/

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/delay.h>
#include "gpio.hpp"
#include "usart.hpp"

using namespace MikroDuino;

/*****************************************************************
**********
Circular Buffer
*****************************************************************
*********/

constexpr uint8_t RX_BUFFER_SIZE = 64;

volatile char rxBuffer[RX_BUFFER_SIZE];

volatile uint8_t rxHead = 0;
volatile uint8_t rxTail = 0;


/*****************************************************************
**********
Buffer Functions
*****************************************************************
*********/

bool bufferAvailable()
{
return (rxHead != rxTail);
}

char bufferRead()
{
char c = rxBuffer[rxTail];

rxTail++;

if(rxTail >= RX_BUFFER_SIZE)
rxTail = 0;

return c;
}


/*****************************************************************
**********
USART Receive Interrupt
```

```
Executed automatically every time a character arrives.
*************************************************************
*********/

ISR(USART_RX_vect)
{
char c = UDR0; // Reading UDR0 clears RX interrupt flag

uint8_t next = rxHead + 1;

if(next >= RX_BUFFER_SIZE)
next = 0;

// Store only if buffer not full
if(next != rxTail)
{
rxBuffer[rxHead] = c;
rxHead = next;
}

// Echo received character immediately
USART0.write(c);
}


/*************************************************************
**********
Main
*************************************************************
*********/

int main()
{
//-----------------------------------------------------
// Configure USART
//-----------------------------------------------------

USARTConfig config;

config.baudRate = 9600;
config.mode = USARTMode::TX_RX;

USART0.begin(config);

//-----------------------------------------------------
// Enable RX interrupt
//-----------------------------------------------------

USART0.enableRxInterrupt();

//-----------------------------------------------------
// Enable global interrupts
//-----------------------------------------------------

sei();
GPIO::output(PB0);

USART0.writeLine("================================");
USART0.writeLine(" MikroDuino USART Interrupt Demo ");
USART0.writeLine("================================");
```

```
USART0.writeLine("Type something and press ENTER.");

//-------------------------------------------------------
// Command buffer
//-------------------------------------------------------

char command[32];
uint8_t index = 0;

while(1)
{
//-------------------------------------------------------
// Process received characters
//-------------------------------------------------------

while(bufferAvailable())
{
char c = bufferRead();

if(c == '\r' || c == '\n')
{
command[index] = '\0';

USART0.writeLine("");
USART0.write("Received: ");
USART0.writeLine(command);

index = 0;
}
else
{
if(index < sizeof(command)-1)
{
command[index++] = c;
}
}
}

//-------------------------------------------------------
// Other application code could execute here
//-------------------------------------------------------
GPIO::toggle(PB0);
_delay_ms(500);
}
}
```